Hoja de ejercicios – Tema 1

Programación Declarativa Aplicada
Máster en Ingeniería Informática
Facultad de Informática - UCM

1. La sucesión de Fibonacci $(f_n)_{n\in\mathbb{N}}$ viene definida por la siguiente recurrencia:

$$\begin{aligned} f_0 &= 0 \\ f_1 &= 1 \\ f_n &= f_{n-1} + f_{n-2} \qquad \text{para } n \geq 2 \end{aligned}$$

   a) Implementa en Elixir una función recursiva `fib/1` que reciba un número entero n y devuelva el valor de f_n utilizando la recurrencia anterior. ¿Cuál es el coste en tiempo de esta función en función del parámetro n?

   b) Un algoritmo imperativo (en Python) para calcular f_n en coste $\mathcal{O}(n)$ es el siguiente:

```
def fib(n):
    i = 0     # Contador desde 0 hasta n
    f0 = 0    # Valor de f_i
    f1 = 1    # Valor de f_{i+1}
    while i < n:
        e = f0 + f1
        f0 = f1
        f1 = e
        i += 1
    return f0
```

   Reescribe la función `fib/1` del apartado anterior utilizando una función auxiliar recursiva de cola y parámetros acumuladores.

2. La implementación de `pow/2` vista en las diapositivas tiene coste lineal con respecto al exponente. Implementa una versión de coste logarítmico utilizando las siguientes propiedades de las potencias:

$$\begin{aligned} x^m &= x^{\frac{m}{2}} * x^{\frac{m}{2}} \qquad \text{para todo } m \text{ par} \\ x^m &= x^{\lfloor \frac{m}{2} \rfloor} * x^{\lfloor \frac{m}{2} \rfloor} * x \qquad \text{para todo } m \text{ impar} \end{aligned}$$

   donde la notación $\lfloor \frac{a}{b} \rfloor$ denota la división entre a y b redondeada hacia abajo.

3. Implementa una función `cuenta_digitos/1` que reciba un número entero n y devuelva el número de dígitos significativos que contiene. Por ejemplo, para $n = 5672$ la función debe devolver 4, y para $n = 0325$ debe devolver 3. Escribe dos soluciones para este problema:

   a) Una de ellas utilizando una función recursiva que no sea de cola.

   b) Otra utilizando una función recursiva de cola, utilizando parámetros acumuladores.

4. Utilizando las funciones de los dos ejercicios anteriores, escribe una función `concatenar_num/2` que devuelva el resultado de concatenar los números pasados como parámetro. Por ejemplo, la expresión `concatenar_num(435, 9384)` debe evaluarse al número `4359384`.

   No está permitido convertir los números a *strings* para realizar este ejercicio.

5. Implementa una función `binario_str/1` que, dado un número *n*, devuelva una cadena con la representación binaria de *n*. Por ejemplo, `binario_str(19)` debe evaluarse a la cadena `"10011"`.

6. Escribe una función `rellenar_con/3` que reciba una cadena *s*, un número *n* y una cadena *c* de un solo carácter. La función debe devolver el resultado de colocar el carácter *c* a la izquierda de *s* las veces que sean necesarias hasta que la cadena resultante tenga longitud *n*. Por ejemplo, la expresión `rellenar_con("34", 5, "-")` debe devolver la cadena `"---34"`.

   Utiliza parámetros acumuladores y recursión de cola.

7. Escribe un *script* en Elixir que adivine un número pensado por el usuario (entre 0 y 100). El programa escribirá un número *X* y el usuario tiene que indicar si el número en el que había pensado es mayor (`M`), menor (`m`) o igual (`=`) que *X*. Tras la respuesta del usuario, el programa tendrá que imprimir otro posible candidato *X*, volver a preguntar al usuario, y así sucesivamente. El programa termina cuando ha acertado el número del usuario, o bien cuando detecta que no existe ningún número consistente con las respuestas que ha dado el usuario.

   Utiliza las funciones `IO.puts/1` y `IO.gets/1` de la biblioteca estándar de Elixir.

   Ejemplo de ejecución 1

```
¿Es el 50?
  '=': sí, el número es 50
  'M': no, mi número es mayor
  'm': no, mi número es menor

Introduce tu opción: m
¿Es el 25?
[...]
Introduce tu opción: M
¿Es el 38?
[...]
Introduce tu opción: =
¡Tu número es 38!
```

   Ejemplo de ejecución 2

```
¿Es el 50?
  '=': sí, el número es 50
  'M': no, mi número es mayor
  'm': no, mi número es menor
Introduce tu opción: m
¿Es el 25?
[...]
Introduce tu opción: m
¿Es el 12?
[...]
Introduce tu opción: M
¿Es el 19?
[...]
Introduce tu opción: m
¿Es el 16?
[...]
Introduce tu opción: M
```

```
¿Es el 18?
[...]
Introduce tu opción: M
¡Me has engañado!
```

8. Recuerda que la función `take(xs, n)` devuelve los `n` primeros elementos de la lista `xs`. Escribe una función `drop(xs, n)` que devuelva los elementos de `xs` *excepto* los `n` primeros.

   Por ejemplo, `drop([:a, :b, :c, :d, :e], 3)` devuelve la lista `[:d, :e]`.

9. En Elixir, una *keyword list* es una lista cuyos elementos son pares, donde la primera componente de cada par es un átomo (denominado *clave*). Por ejemplo:

   ```
   [{:nombre, "Gerardo"}, {:edad, 41}, {:pais, "UK"}]
   ```

   Elixir ofrece una sintaxis alternativa para representar este tipo de listas:

   ```
   [nombre: "Gerardo", edad: 41, pais: "UK"]
   ```

   Implementa dos funciones:

   - `keyfind(xs, k)`. Dada una *keyword list* `xs`, devuelve la segunda componente del par cuya clave es `k`. Por ejemplo, si `xs` es la lista anterior, `keyfind(xs, :edad)` devuelve `41`. Si no existe ningún par con la clave proporcionada, la función debe devolver `nil`.
   - `keytake(xs, k)`. Dada una *keyword list* `xs` y una clave `k`, devuelve una tupla con dos componentes. La primera componente es el par `{k, v}` correspondiente de la *keyword list*, o `nil` si no existe ningún par con la clave proporcionada. La segunda componente es el resultado de eliminar el par `{k, v}` de la *keyword list* `xs`, si existe. Por ejemplo, si `xs` es la lista anterior, `keytake(xs, :edad)` devuelve `{{:edad, 41}, [nombre: "Gerardo", pais: "UK"]}`.

   Modifica la función `keyfind/2` para que reciba un tercer parámetro `d`, cuyo valor por defecto es `nil`. En el caso en el que la clave a buscar no se encuentre en la *keyword list*, la función debe devolver `d`.

10. Escribe una función `ordenada?/1` que, dada una lista, devuelva `true` si la lista tiene los elementos en orden ascendente, o `false` en caso contrario.

11. En este ejercicio vamos a implementar el algoritmo de ordenación por inserción. Para ello:

    - Escribe una función `insertar_ord(xs, x)` que, dada una lista `xs` ordenada de manera ascendente, devuelva el resultado de insertar `x` en `xs` de modo que la lista resultante siga estando ordenada.
    - Utilizando la función anterior, escribe una función `insert_ord(xs)` que devuelva el resultado de ordenar la lista `xs` mediante inserciones ordenadas sucesivas de los elementos de la lista original.

12. En este ejercicio vamos a implementar el algoritmo de ordenación *Quicksort*. Para ello:

    - Escribe una función `particion(xs, p)` que, dada una lista `xs` y un valor `p`, devuelva un par de listas `{ys, zs}` donde `ys` contiene los elementos de `xs` menores o iguales que `p` y `zs` contiene aquellos que sean mayores que `p`.
    - Utilizando la función anterior, escribe una función `qsort(xs)` que devuelva el resultado de ordenar `xs` utilizando el algoritmo *Quicksort*.

13. En este ejercicio vamos a implementar al algoritmo de ordenación por mezclas (*Mergesort*). Para ello

    - Escribe una función `repartir(xs)` que devuelva un par de listas, donde la primera lista contiene los elementos de `xs` que ocupan posiciones impares y la segunda lista contiene aquellos que ocupan posiciones pares. Por ejemplo, `repartir([:b, :c, :a, :d])` devuelve el par `{[:b, :a], [:c, :d]}`.
    - Escriba una función `mezclar(xs, ys)` que, dadas dos listas ordenadas `xs` e `ys`, devuelva otra lista ordenada con los elementos de ambas.
    - Utilizando las dos funciones anteriores, implementa una función que ordene el vector pasado como parámetro mediante el algoritmo de ordenación por mezclas.

14. Las funciones `all?/2` y `any?/2` reciben una lista `xs` y una función `f`. Devuelven `true` si `f(x)` toma el valor *cierto* para *todos* los elementos `x` (en el caso de `all?`) o para *algún* elemento `x` (en el caso de `any?`) de la lista `xs`.

    a) Implementa `all?/2` y `any?/2` utilizando recursión explícita.

    b) Implementa `all?/2` y `any?/2` utilizando la función `reduce`.

15. Sin utilizar recursión explícita, escribe una función `ordenada?/1` que diga si la lista pasada como parámetro está ordenada de manera ascendente.

16. La función `count/2`, dada una lista `xs` y un valor `x`, indica cuántos elementos de `xs` son iguales a `x`.

    a) Implementa `count/2` utilizando recursión explícita.

    b) Implementa `count/2` utilizando `reduce`.

    c) Modifica la implementación anterior haciendo que `count/2`, en lugar de un valor `x`, reciba una función `f`. De este modo, `count/2` debe devolver el número de elementos `x` de la lista de entrada para los cuales `f` devuelve el valor *cierto*.

17. Implementa la función `filter/2` utilizando `flat_map/2`, y sin utilizar recursión explícita.

18. Como recordarás, la función `reduce/3` realiza un recorrido de la lista de entrada, manteniendo un acumulador que se propaga de un elemento al siguiente. La lista siempre se recorre en su totalidad.

    Existe una variante de la función `reduce/3`, llamada `reduce_while/3`, que permite detener el recorrido en un punto intermedio de la lista. En este caso, la función `f` que recibe `reduce_while/3` como parámetro puede devolver:

    - `{:cont, v}`, que indica que el recorrido debe continuar, de modo que el valor *v* se propagará como acumulador al siguiente elemento de la lista.
    - `{:halt, v}`, que indica que el recorrido debe detenerse. En ese caso, la función `reduce_while/3` devolverá directamente el valor *v* devuelto.

    Se pide:

    a) Implementa la función `reduce_while/3` utilizando recursión explícita.

    b) Implementa la función `all?/2` del ejercicio 14 utilizando `reduce_while/3`.

19. La función `frequencies/1`, dada una lista `xs`, devuelve un diccionario que asocia cada elemento de `xs` con el número de veces que aparece en dicha lista. Por ejemplo, dada la lista `[:a, :b, :a, :c, :c, :a]`, la función `frequencies/1` devolvería el diccionario `%{a: 3, b: 1, c: 2}`. Implementa la función `frequencies/1` sin hacer uso de recursión explícita.

20. Las funciones `take_while/2`, `drop_while` y `split_while/2` reciben una lista `xs` y una función `f`.

    - `take_while/2` devuelve los elementos desde el principio de la lista `xs` hasta llegar al primer elemento `x` para el que la función `f(x)` devuelve *falso*. El elemento `x` no se incluye en la lista resultado.
    - `drop_while/2` devuelve el primer elemento `x` para el que la función `f(x)` devuelve *falso*, y todos los situados después de él.
    - `split_while/2` devuelve una tupla `{ts, ds}`, donde `ts` es el resultado de `take_while/2` y `ds` es el resultado de `drop_while/2`.

    Implementa estas tres operaciones.

21. Suponemos el siguiente tipo unión para representar árboles binarios:

$$arbol ::= arbol_vacio \mid nodo(arbol, elem, arbol)$$

    donde *elem* es el tipo de los elementos del árbol.

    a) Define el tipo *arbol* en Elixir mediante *structs* e implementa el protocolo `String.Chars` para este tipo.

    b) Escribe una función `map` que reciba un árbol binario y una función `f`, y devuelva el árbol binario que resulta de aplicar `f` a los valores contenidos en los nodos del árbol pasado como parámetro.

    c) Supongamos la siguiente definición de la función `reduce_tree` sobre árboles:

$$\begin{aligned} reduce_tree(arbol_vacio, i, f) &= i \\ reduce_tree(nodo(iz, x, dr), i, f) &= f(reduce_tree(iz, i, f), x, reduce_tree(dr, i, f)) \end{aligned}$$

    Implementa la función `reduce_tree` en Elixir. Utilizando esta función:

    1) Implementa una función `sum(t)` que devuelva la suma de los valores contenidos en los nodos del árbol `t`.
    2) Implementa un método `height(t)` que devuelva la altura del árbol `t`.
    3) Implementa un método `balanced?(t)` que devuelva un booleano indicando si `t` está equilibrado o no. Un árbol binario está equilibrado si es el árbol vacío, o es un árbol en el que la diferencia de alturas entre sus hijos es menor o igual a 1, y ambos hijos están equilibrados.

22. Implementa una función `map_option(xs, f)` que se comporte de manera similar a `map`, pero suponiendo que la función `f` recibida como parámetro devuelve una tupla `{:ok, v}` en caso de éxito, u otro valor en caso de fallo. En este último caso, la función `map_option` debe devolver el primer fallo producido. En caso de éxito, la función `map_option` debe devolver una tupla `{:ok, ys}`, donde `ys` contiene los valores devueltos por `f`.

23. Implementa una función `reduce_option(xs, ac, f)` que realice un recorrido de una lista con un valor acumulador, de manera similar a la función `reduce`, pero que interrumpa el recorrido si uno de los valores intermedios tomados por el acumulador es un valor distinto de `{:ok, v}` para algún `v`.

24. Supongamos una cuadrícula de `MAP_SIZE` $\times$ `MAP_SIZE` como la mostrada en la Figura 1. Cada casilla se identifica mediante un par $\{x, y\}$, donde $\{1, 1\}$ representa la casilla situada más al suroeste:

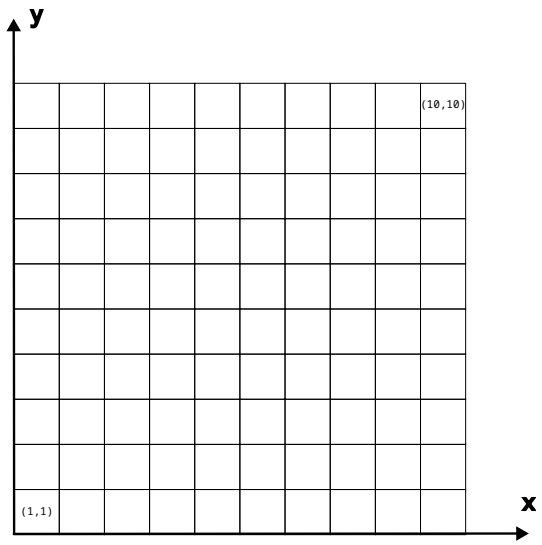


Figura 1: Cuadrícula con `MAP_SIZE = 10`

Un robot comienza en la posición $\{1, 1\}$ y realiza una serie de movimientos. Cada movimiento consiste en saltar a una casilla adyacente en una dirección determinada. Representamos las direcciones mediante un tipo unión:

$$\textit{direccion} ::= \textit{norte} \mid \textit{sur} \mid \textit{este} \mid \textit{oeste}$$

En este caso, el valor *norte* indica un movimiento en el sentido positivo del eje y, mientras que *este* indica un movimiento en el sentido positivo del eje x.

Implementa una función `move` que, dada una posición inicial y una lista de direcciones, determine si el robot saldría de los límites de la cuadrícula al realizar secuencialmente la lista de movimientos indicada y, en caso de no salirse, devuelva cuál sería la posición final alcanzada por el robot.

25. El formato JSON (*JavaScript Object Notation*) es un formato de intercambio de datos muy utilizado en aplicaciones web. Utiliza una notación muy similar a la del lenguaje JavaScript, de ahí su nombre. Los valores JSON están formados por:

    - Literales de tipo numérico. Por ejemplo: `23`, `-2.5`,
    - Literales de tipo cadena de caracteres. Los caracteres de la cadena están delimitados por comillas, y es posible tener secuencias de escape (`\n`, `\"`, etc.). Por ejemplo: "`Daniela`", "`Esto es\nuna prueba`",
    - Literales de tipo booleano: `true` o `false`,
    - Literal `null`,
    - Arrays, cuyos elementos pueden ser cualquier valor JSON. Por ejemplo: `[3, "Javier", true]`, `[[1, false], [3, 2, true], false]`.
    - Objetos, que son conjuntos de pares clave/valor. Las claves son necesariamente literales de tipo cadena de caracteres. Los valores pueden ser cualquier elemento JSON. Por ejemplo: `{ "nombre": "Diana", "edad": 23, "fechaNacimiento": [23, 5, 2012] }`

    Se pide:

a) Definir un tipo unión en Elixir mediante *structs* que representen cada una de las alternativas.

b) Definir implementaciones del protocolo `String.Chars` para cada uno de los *structs* definidos anteriormente. Las cadenas devueltas por estas implementaciones deben adecuarse al formato JSON.

26. Supongamos un fichero con información de las personas participantes en un estudio médico:

Nombre	Edad	Fumador/a	Provincia
Javier Francisco Peña	35	SI	Ávila
Marta Delgado	42	SI	Cáceres
Gloria Benito	31	NO	Madrid
...	...	...	...

La información se encuentra en formato CSV (*Comma Separated Values*). Cada fila contiene la información de una persona, excepto la primera fila, que contiene la cabecera de la tabla:

```
Nombre,Edad,Fumador,Provincia
Javier Francisco Peña,35,SI,Avila
Marta Delgado,42,SI,Cáceres
Gloria Benito,31,NO,Madrid
...
```

Escribe una función en Elixir que, dado el nombre del fichero en el que se encuentra la información mencionada anteriormente, calcule la media de las edades de las diez primeras personas fumadoras que aparezcan en el fichero.

```
def media_edades_fumadores(nombre_fichero, valor_defecto)
```

Si hay menos de diez personas fumadoras en el fichero, deberá devolverse la media de las personas fumadoras que haya en él. Si no hay ninguna persona fumadora en el fichero, deberá devolverse `valor_defecto`. La función debe ser perezosa sobre el parámetro `valor_defecto`.

Indicación. Utiliza el paquete CSV[1].

27. La función exponencial e^x puede calcularse mediante la siguiente fórmula:

$$e^x = \sum_{i=0}^{+\infty} \frac{x^i}{i!} = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots$$

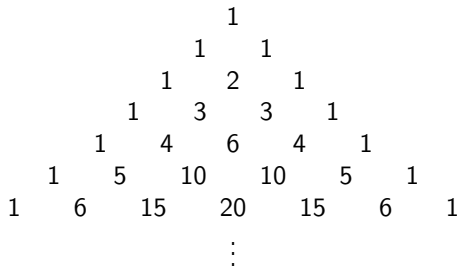
Escribe una función `exp(x, n)` que devuelva el valor de e^x utilizando los n primeros términos del sumatorio. Para ello no puedes utilizar ni bucles, ni recursión.

Indicación. Crea dos *stream*: uno de ellos con los valores de las potencias $x^0, x^1, x^2, \dots$ y otro con los valores de los factoriales $0!, 1!, 2!, \dots$. Combina ambos mediante `Stream.zip_with/3`.

28. En matemáticas, una *terna pitagórica* es una tupla (a, b, c) tal que a, b y c son números enteros que cumplen $a^2 + b^2 = c^2$. Utilizando listas intensionales, escribe una función `ternas(n)` que devuelva las ternas pitagóricas tales que $0 \leq a, b, c \leq n$.

[1] https://hexdocs.pm/csv/CSV.html

29. El *Triángulo de Pascal* es un conjunto de números enteros dispuestos en forma de triángulo, donde cada número es igual a la suma de los dos números que están encima de él, suponiendo que la cúspide y los laterales del triángulo contienen el número **1**.



Escribe una función `comb(i, j)` que devuelva el j-ésimo elemento de la i-ésima fila del triángulo.

30. Dado un tablero de ajedrez de tamaño $n \times n$, queremos saber las formas posibles en las que podemos colocar n reinas sin que haya dos reinas que se ataquen entre ellas (es decir, que estén colocadas en la misma fila, columna, o diagonal). Para ello escribe una función `reinas(n)` que devuelta un *stream* con las posibles soluciones al problema. Cada solución es una lista de n pares que contienen las posiciones de las reinas dentro del tablero.